

Лабораторная работа №6: Многошаговая регистрация

Цель работы:

Создать форму регистрации, состоящую из 3-х шагов, используя `useReducer` для управления всеми данными и валидацией `Zod`.

Требования к реализации:

1. Структура состояния (State)

Ваш стейт должен содержать:

- `currentStep`: номер текущего шага (1, 2 или 3).
- `formData`: объект с полями (`email`, `password`, `username`, `city`, `occupation`).
- `errors`: объект, где ключи — названия полей, а значения — текст ошибки.
- `isSubmitting`: флаг процесса отправки.

2. Шаги формы:

- **Шаг 1 (Аккаунт):** Поля `email` и `password`.
 - *Валидация:* Email через `Regex`, пароль минимум 8 символов.
- **Шаг 2 (Профиль):** Поля `username` и `city`.
 - *Валидация:* Поля не должны быть пустыми.
- **Шаг 3 (О себе):** Поле `occupation` (выбор из списка) и галочка "Согласен с правилами".

3. Логика Reducer:

Редьюсер должен обрабатывать следующие типы действий (Actions):

- `UPDATE_FIELD`: обновляет конкретное поле в `formData` и очищает ошибку для этого поля.
- `SET_ERROR`: записывает ошибку для конкретного поля.
- `NEXT_STEP`: переходит к следующему шагу (только если текущий валиден).
- `PREV_STEP`: возвращает на шаг назад.
- `SUBMIT_START` / `SUBMIT_SUCCESS`: управление состоянием отправки.

4. Использовать контролируемые компоненты

Задание:

1. **Типизация:** Опишите интерфейс `IFormState` и `TFormAction`.
2. **Редьюсер:** Напишите функцию `registrationReducer`. Используйте оператор `switch`.
3. **Компонент:** * Создайте основной компонент `RegistrationForm`.
 - Разделите `TSX` на части (можно через `renderStep1()`, `renderStep2()` или отдельные компоненты).
 - Добавьте проверку: кнопка "Далее" не должна пускать на следующий шаг, если есть ошибки валидации.

4. **Финальный Submit:** На 3-м шаге при клике "Зарегистрироваться" симитируйте задержку в 2 секунды (`setTimeout`) и выведите данные формы в консоль.

Требования по использованию Zod

При реализации валидации необходимо придерживаться следующих правил:

1. "Single Source of Truth" (Единый источник истины)

- **Запрещено** вручную описывать интерфейс `IFormData`. Он должен автоматически выводиться из Zod-схемы с помощью `z.infer<typeof RegistrationSchema>`.
- Все текстовые сообщения об ошибках (например, *"Пароль слишком короткий"*) должны быть описаны внутри схемы Zod.

2. Схемы для каждого шага

- Создай одну базовую схему `RegistrationSchema`, описывающую все поля.
- Для валидации конкретных шагов используй методы `.pick()`, чтобы извлекать только нужные поля (например, для Шага 1: `.pick({ email: true, password: true })`).

3. Безопасная валидация

- Внутри компонента используй метод `.safeParse()`, чтобы избежать выбрасывания исключений (Runtime Errors).
- Результат валидации должен преобразовываться в объект ошибок, ключи которого соответствуют полям в `IFormState['errors']`.

Контрольные вопросы:

1. Опиши четыре роли паттерна Flux. Какая из них является «центральным узлом», рассылающим уведомления?
2. Что означает принцип «однонаправленного потока данных»? Может ли Store во Flux быть изменен напрямую из View?
3. Назови «три кита» `useReducer`. Без каких составляющих работа этого хука невозможна?
4. Что такое Reducer с точки зрения функционального программирования? Почему важно, чтобы это была «чистая функция»?
5. Какую роль выполняет функция `dispatch`?
6. Как типизация `Action` в TypeScript помогает избежать ошибок при «бомбардировке» редьюсера командами?
7. Как редьюсер обрабатывает экшены, тип которых ему неизвестен (ветка `default`)?
8. Чем метод `.safeParse()` отличается от `.parse()` и в каких случаях какой использовать?
9. Как с помощью Zod создать тип на основе существующей схемы без ручного написания `interface`?
10. Зачем использовать `.pick()` или `.partial()` при создании схем для API?